River Learning: Forward-Only Path Carving Without Backpropagation

Yeonseong Cynn
whitepep@gmail.com
2026

Abstract

We present *River Learning*, a forward-only training algorithm grounded in a single principle: given a source (input) and a sink (output), flow finds the path. No gradient, no backward pass, and no global objective are required — only the knowledge of where information enters and where it must arrive. We introduce the concept of **Natural Optimum**: the optimal solution is not computed or designed — it is carved by flow alone. As a river does not calculate the shortest path to the sea but simply flows until it breaks through, a Natural Optimum network requires only a source, a sink, and the patience to let flow do its work. Critically, **inference and learning occur within the same forward pass**: there is no separate training phase, no stored computation graph, and no mode switch between learning and deployment. The network improves on every input it processes. We demonstrate that this procedure learns AND, OR, NOR, and XOR to 100% accuracy, and conjecture that any finite directed network problem with a defined source and sink is in principle solvable by this mechanism.

1 Introduction

Modern deep learning relies almost exclusively on backpropagation [6], which requires a differentiable loss function, a stored forward computation graph, and a backward pass that propagates error signals layer by layer in the reverse direction. This dependence on backward computation is both a practical constraint (memory, non-local weight updates) and a biological implausibility [1, 5].

Several alternatives have been proposed, including Hebbian learning [2], forward-forward [3], and target propagation [4]. However, these methods still require auxiliary objectives or layer-wise targets derived from global supervision.

We reduce the learning problem to its irreducible minimum:

> *Know where it enters. Know where it must arrive. Let it flow. Optimization is not computed — it is found.*

This is what we call **Natural Optimum**: the optimal path is not designed, not gradient-descended into, not searched for — it emerges from the physics of flow through a resistive medium. No loss surface, no gradient, no backward pass. Only a source, a sink, and a medium that erodes under flow.

River Learning formalizes this principle. The contributions of this paper are:

- A forward-only learning algorithm with no backward pass and no global objective function.
- **Unified inference and learning**: the same forward pass that produces a prediction also performs the resistance update. There is no training phase, no inference phase — only a single operation that does both.
- Empirical demonstration on Boolean functions (AND, OR, NOR, XOR, Parity) with 100% accuracy.
- A conjecture that any finite directed network with a defined source and sink is learnable by this mechanism, with a sketch of the formal path toward proof.

It takes its name and intuition from *natural intelligence*: the capacity of physical systems — rivers, vascular networks, slime molds — to discover optimal transport paths through local erosion alone, without any global objective or backward signal [7]. A river does not compute the gradient of the landscape. It starts at the source, ends at the sea, and the canyon forms by itself.

We conjecture that this framing generalizes: *any problem that can be expressed as a directed network with a defined source and sink is in principle learnable by flow.* Boolean functions are the first test case.

Routing, scheduling, and inference on graphs are natural extensions. The only feedback used is a scalar binary signal— *correct* or *incorrect*—available at the output after the forward pass. No gradients are computed. No backward pass is taken. The network modifies itself by reducing resistance along the paths that were active during an incorrect prediction, making those paths more likely to fire differently in the future.

The metaphor is a river carving a canyon: water does not compute the gradient of the landscape; it simply flows, and the path that flows most carves deepest.

Nature has been doing this all along. We did not invent the mechanism — we named it.

The consequences follow directly from the metaphor:

- A path is carved forward — never backward. *(forward-only)*
- Flow erodes as it moves — inference and learning are one act. *(no phase separation)*
- No force is needed; the canyon forms by itself. *(Natural Optimum)*
- The same terrain always carves the same canyon. *(topological convergence)*

2 Method

2.1 Network Structure

The network has two hidden layers connected by three resistance matrices:

$$\text{Input}(n_{\text{in}}) \xrightarrow{D_{12}} \text{H1}(n_{h1}) \xrightarrow{D_{23}} \text{H2}(n_{h2}) \xrightarrow{D_{34}} \text{Output}(2)$$

Each connection layer is parameterized by a *resistance matrix* $D \in \mathbb{R}_{>0}^{m\times n}$. All resistances are initialized to $d_0 = 100$ plus small uniform noise. Each hidden layer also has a threshold vector $\boldsymbol{\theta} \in \mathbb{R}^n$. Each input x_i is routed through two complementary channels $(x_i,\, 1-x_i)$: when one carries no signal, the other does. This ensures a baseline flow is always present at every input, regardless of its value. The effective input dimension is $2n_{\text{in}}$.

2.2 Forward Pass

From a resistance matrix D, conductance is computed as:

$$C_{ij} = \min\left(\frac{1}{\max(d_{ij},\, \epsilon)},\, \frac{d_0}{\max(d_{ij},\, 1)}\right)$$

where $\epsilon \ll 1$ prevents division by zero. C_{ij} is large when resistance is small (*carved* path) and small when resistance is large (*blocked* path); $d_0/\max(d_{ij}, 1)$ caps conductance at d_0 when $d_{ij} \le 1$.

Given encoded input $\tilde{\mathbf{x}} \in \{0,1\}^{2n_{\text{in}}}$:

$$\begin{aligned} \mathbf{a}^{(1)} &= \tilde{\mathbf{x}} \cdot C_{12}, & f_j^{(1)} &= \mathbf{1}[a_j^{(1)} > \theta_j^{(1)}] \\ \mathbf{a}^{(2)} &= \mathbf{f}^{(1)} \cdot C_{23}, & f_k^{(2)} &= \mathbf{1}[a_k^{(2)} > \theta_k^{(2)}] \\ \mathbf{s} &= \mathbf{f}^{(2)} \cdot C_{34}, & \hat{y} &= \arg\max_c\, s_c \end{aligned}$$

2.3 Carving Rule

When $\hat{y} \neq y$ (incorrect prediction), the network *carves* the active path toward the correct output class y:

Algorithm 1 Canyon Carving (one sample)

1: $h_2 \leftarrow$ random node in $\{k : f_k^{(2)} = 1\}$
2: $d_{h_2,y} \leftarrow \max(1,\ d_{h_2,y} - \alpha)$
3: $h_1 \leftarrow$ random node in $\{j : f_j^{(1)} = 1\}$
4: $h_2' \leftarrow$ random node in H_2 {exploration target, independent of h_2}
5: $d_{h_1,h_2'} \leftarrow \max(1,\ d_{h_1,h_2'} - \alpha)$
6: $i \leftarrow$ random active input ($x_i = 1$)
7: $h_1' \leftarrow$ random node in H_1 {exploration target, independent of h_1}
8: $d_{i,h_1'} \leftarrow \max(1,\ d_{i,h_1'} - \alpha)$

Here $\alpha > 0$ is the carving amount (erosion rate). Resistance is clamped at a minimum of 1 to prevent collapse. No carving occurs on correct predictions.

2.4 Sleep-Recovery

Without a recovery mechanism, all resistances would eventually reach their minimum, saturating the network. After each step, if the mean saturation $1-\bar{d}/d_0$ exceeds a threshold (0.55), a *sleep* step uniformly increases all resistances by a small amount β, clipped at d_0:

$$d_{ij} \leftarrow \min(d_{ij} + \beta,\ d_0)$$

This models synaptic consolidation during sleep: weakly carved paths fade, strongly carved paths survive.

2.5 Threshold Adaptation

Every 10 steps, firing thresholds are updated to the midpoint of the observed activation range, for each hidden layer $\ell \in \{1, 2\}$:

$$\theta_j^{(\ell)} \leftarrow \frac{\min_s a_{s,j}^{(\ell)} + \max_s a_{s,j}^{(\ell)}}{2}$$

This ensures nodes in both layers discriminate rather than fire uniformly.

2.6 Training Procedure

Training runs for up to E epochs and R random restarts. The best-scoring network across restarts is retained. Hyperparameters: $d_0 = 100$, $\alpha = 12$, $\beta = 3$, saturation threshold $= 0.55$, $E = 1000$, $R = 50$.

3 Experiments

3.1 Boolean Gate Learning

We train the network on four fundamental Boolean functions. Each experiment is repeated with 5 different random seeds; mean accuracy is reported.

Table 1: Boolean gate learning accuracy (mean over 5 seeds, $n_{h1} = n_{h2} = 8$, epochs=1000, restarts=50, $\alpha = 12$)

Function	inputs	Samples	Accuracy
AND	2	4	1.000
OR	2	4	1.000
NOR	2	4	1.000
XOR	2	4	1.000

XOR is the critical case: it is not linearly separable, requires a hidden layer to represent, and has been the standard benchmark for non-trivial learning since [6]. River Learning solves it without a single gradient computation.

3.2 Parity Functions

n-bit parity is a demanding test: every bit is necessary, no input can be ignored, and the function is not decomposable into simpler pieces solved by a single layer.

Table 2: Parity learning accuracy†

Task	n	Samples	Accuracy
Parity-2	2	4	1.000
Parity-4	4	16	1.000
Parity-8	8	256	1.000

† Parity-n uses $n_h = 16$ hidden nodes per layer, reflecting the combinatorial growth of the input space (2^n samples for n-bit parity). All other hyperparameters (α, β, saturation threshold, epochs, restarts) are identical to Table 1.

All parity functions are learned to 100% accuracy under these settings, confirming that forward-only carving can handle multi-layer, non-decomposable functions.

4 Discussion

Why no backward pass is needed. In standard backpropagation, the gradient $\partial\mathcal{L}/\partial w_{ij}$ tells each weight exactly how to change. In River Learning, the carving rule instead identifies an *active path* that contributed to an incorrect output and reduces resistance along it, making that path stronger for the correct class. The signal is local (which path fired?) and binary (correct or not?), yet it suffices to learn any function tested here.

Physical interpretation and Natural Optimum. The network is a resistance graph. Conductance accumulates at paths that fire correctly. Sleep prevents global saturation. This is precisely how water carves a canyon: flow erodes rock, the carved path deepens, other paths receive less flow and fill in. No global objective is minimized; the optimal path emerges. This is Natural Optimum — not an optimization procedure, but an optimization outcome. The system does not know it is optimizing anything. It only flows. The optimum is what remains after the flow. A network initialized with 16 nodes converges to 3 for AND — the same minimum every time, from any starting point. The function decided the architecture. No hyperparameter search required.

No separation between inference and learning. In all gradient-based systems, learning and inference are fundamentally distinct operations: training requires a stored computation graph and a backward pass, while inference is a read-only forward pass on a frozen network. River Learning eliminates this distinction. The same forward pass that computes the output also identifies the active path — and the carving step that follows uses only information already present at inference time (which nodes fired, and whether the output was correct). No additional memory, no mode switch, and no recomputation are required. A deployed River network is always learning. This property is not an engineering convenience — it is a structural consequence of the algorithm, and it has implications for continual learning, online adaptation, and the design of physical substrates where training and inference circuits cannot be separated.

Comparison with Forward-Forward. Hinton's Forward-Forward algorithm [3] also avoids backpropagation, but requires two forward passes (positive and negative data) and a layer-wise goodness objective. River Learning requires only one forward pass and a single binary output signal.

Scope of the conjecture. River Learning requires only three things from a problem: (1) a defined input space (source), (2) a defined output space (sink), and (3) a way to evaluate whether a given path produced a correct result. Any finite, discrete-input directed network that satisfies these conditions — Boolean circuits, routing graphs, sequence transducers, inference chains — is a candidate for flow-based learning. The resistance medium and the carving rule are the only mechanism. The path structure needed to solve the problem emerges without being designed. Formal support for this conjecture can be approached via two steps: first, noting that $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOR}\}$ is a functionally complete basis, so any Boolean function is expressible in this network; second, a Lyapunov argument over the finite resistance state space suggests convergence in a bounded number of carving steps. A rigorous proof is left for future work.

On random restarts. Random restarts ($R = 50$) are a practical acceleration, not a theoretical requirement. A network initialized with 16 hidden nodes converges to 3 effective nodes for AND — the same minimal structure reached from any starting point, regardless of initialization. Given sufficient iterations, a single run reaches the same optimum. Restarts compress what flow would eventually complete on its own.

Open questions. Formal convergence bounds and the precise boundary of solvable problem classes remain open questions.

5 Conclusion

We have shown that a multi-layer network can learn Boolean functions— AND, OR, NOR, and XOR at 100% accuracy, and n-bit parity up to $n = 8$ under a separate configuration (Table 2)—using only forward passes and a binary correctness signal. No gradient, no backward pass, and no auxiliary objective is required. The method is grounded in a physical analogy (river canyon erosion) and achieves perfect accuracy on all tested functions.

This establishes forward-only path carving as a viable alternative to backpropagation for the class of functions studied, and opens the question of how far this principle extends to larger, continuous-domain tasks.

The missing element in neural learning was not a more complex gradient — it was simply the definition of a sink.

Patent and Collaboration

The methods described in this paper are covered by Korean patent application **10-2026-0052624** (filed 2026), which includes claims on forward-only path carving, threshold-inversion routing, and flow-based competitive suppression. PCT international filing is planned.

The author welcomes:

- **Academic collaboration** — co-authorship on peer-reviewed journal submissions extending this work is open and encouraged.
- **Commercial proposals** — licensing, applied research, or product development based on River Learning or Natural Optimum.

Contact: Yeonseong Cynn `whitepep@gmail.com`

References

[1] Francis Crick. The recent excitement about neural networks. *Nature*, 337(6203):129–132, 1989.

[2] Donald O Hebb. *The Organization of Behavior*. Wiley, 1949.

[3] Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

[4] Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. *Machine Learning and Knowledge Discovery in Databases*, pages 498–515, 2015.

[5] Timothy P Lillicrap, Adam Santoro, Luke Marris, Colin J Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

[6] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[7] Atsushi Tero, Seiji Takagi, Tetsu Saigusa, Kentaro Ito, Dan P Bebber, Mark D Fricker, Kenji Yumiki, Ryo Kobayashi, and Toshiyuki Nakagaki. Rules for biologically inspired adaptive network design. *Science*, 327(5964):439–442, 2010.

Upcoming: River Learning Series

Natural Optimum Series

This paper is the first in a series exploring River Learning and its consequences:

Paper 1 Forward-Only Path Carving Without Backpropagation *(this paper)*
Can a network learn without ever looking backward?

Paper 2 Inference Is Learning: No Phase Separation in Forward-Only Networks
Training mode and inference mode are the same operation. A formal account of what this means for continual learning, online deployment, and physical substrates.

Paper 3 Dual Graph Routing and Competitive Suppression
How paths compete and highways emerge from flow alone.

Paper 4 Topological Convergence: Same Function, Same Structure
Independent networks trained on the same function converge to the same firing pattern — and that pattern is the explanation. A structural foundation for XAI.

Paper 5 Template Sharing and O(1) Scaling
Same function, same highway: learn one prototype and copy. 64-bit parity in the time it takes to learn 2-bit.

Paper 6 Flow Conservation as Emergent Competitive Inhibition
Kirchhoff's current law applied to learning: when one path grows stronger, all others grow weaker — for free.

Paper 7 Automatic Layer Compression and Minimal Representation
Why does this network have exactly this many layers? Because the function decided — not the engineer. Over-provisioned networks self-compress to the irreducible complexity of their target function. Architecture search and hyperparameter tuning are heuristics for a question that has a natural answer.

Paper 8 Function Approximation in Topology Space
Two functions with similar firing patterns are substitutable. Topology is a metric on functions — and distance bounds approximation error.

Coming soon. One paper at a time.

Appendix: Reference Implementation

The listing below is a self-contained Python implementation (NumPy only) that reproduces Table 1.

```
import numpy as np

def _cond(d, init_w=100.0):
    return np.minimum(1.0 / np.maximum(d, 0.01),
                      init_w / np.maximum(d, 1.0))

def dual_rail(data):
    result = []
    for x, y in data:
        xd = []
        for xi in x:
            xd += [xi, 1 - xi]
        result.append((xd, y))
    return result

class RiverNet:
    def __init__(self, n_in, n_h=8, n_out=2, init_w=100.0):
        self.n_in, self.n_h, self.n_out = n_in, n_h, n_out
        self.iw = init_w
        self._reset()

    def _reset(self):
        iw = self.iw
        r = lambda a, b: np.ones((a, b)) * iw + np.random.rand(a, b) * 20
        self.d12 = r(self.n_in, self.n_h)
        self.d23 = r(self.n_h,  self.n_h)
        self.d34 = r(self.n_h,  self.n_out)
        self.t1  = np.random.rand(self.n_h) * 0.5 + 0.1
        self.t2  = np.random.rand(self.n_h) * 0.5 + 0.1

    def _forward(self, X):
        C12 = _cond(self.d12, self.iw)
        C23 = _cond(self.d23, self.iw)
        C34 = _cond(self.d34, self.iw)
        a1 = X @ C12;               f1 = a1 > self.t1
        a2 = f1.astype(float) @ C23; f2 = a2 > self.t2
        return a1, f1, a2, f2, f2.astype(float) @ C34

    def score(self, data):
        X = np.array([x for x, _ in data], dtype=float)
        Y = np.array([y for _, y in data])
        *_, s = self._forward(X)
        return float(np.mean(np.argmax(s, 1) == Y))

    def _carve(self, data, amt=12.0):
        X = np.array([x for x, _ in data], dtype=float)
        Y = np.array([y for _, y in data])
        a1, f1, a2, f2, s = self._forward(X)
        for i in np.where(np.argmax(s, 1) != Y)[0]:
            y   = Y[i]
            ai  = np.where(X[i] > 0)[0]
```

```
            f1i = np.where(f1[i])[0]
            f2i = np.where(f2[i])[0]
            if len(f2i):
                self.d34[f2i[np.random.randint(len(f2i))], y] = max(1.,
                self.d34[f2i[np.random.randint(len(f2i))], y] - amt)
            if len(f1i):
                h1 = f1i[np.random.randint(len(f1i))]
                self.d23[h1, np.random.randint(self.n_h)] = max(1.,
                self.d23[h1, np.random.randint(self.n_h)] - amt)
            if len(ai):
                ii = np.random.choice(ai)
                self.d12[ii, np.random.randint(self.n_h)] = max(1.,
                self.d12[ii, np.random.randint(self.n_h)] - amt)

    def _tune(self, data):
        X = np.array([x for x, _ in data], dtype=float)
        a1, _, a2, _, _ = self._forward(X)
        for a, t in [(a1, self.t1), (a2, self.t2)]:
            lo, hi = a.min(0), a.max(0)
            m = hi > lo
            t[m] = (lo[m] + hi[m]) / 2

    def train(self, data, epochs=1000, restarts=50, erode=12.0):
        best, snap = 0.0, None
        for _ in range(restarts):
            self._reset()
            for ep in range(epochs):
                sat = 1 - np.mean(np.concatenate(
                    [d.ravel() for d in
                     [self.d12, self.d23, self.d34]])) / self.iw
                if sat > 0.55:
                    for d in [self.d12, self.d23, self.d34]:
                        np.minimum(d + 3., self.iw, out=d)
                self._carve(data, erode)
                if ep % 10 == 0:
                    self._tune(data)
            sc = self.score(data)
            if sc > best:
                best = sc
                snap = (self.d12.copy(), self.d23.copy(),
                        self.d34.copy(), self.t1.copy(), self.t2.copy())
            if best >= 1.0:
                break
        if snap:
            self.d12, self.d23, self.d34, self.t1, self.t2 = snap
        return best

# -- reproduce Table 1 ------------------------------------------
AND = [([0, 0], 0), ([0, 1], 0), ([1, 0], 0), ([1, 1], 1)]
OR  = [([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 1)]
NOR = [([0, 0], 1), ([0, 1], 0), ([1, 0], 0), ([1, 1], 0)]
XOR = [([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)]

for label, data_raw in [('AND', AND), ('OR', OR),
```

```
                    ('NOR', NOR), ('XOR', XOR)]:
data = dual_rail(data_raw)
accs = []
for seed in [42, 123, 7, 99, 0]:
    np.random.seed(seed)
    net = RiverNet(n_in=len(data[0][0]), n_h=8)
    accs.append(net.train(data))
print(f'{label}  acc={np.mean(accs):.3f}')
```